

Agent Infra · 新智基座

AgentCorp

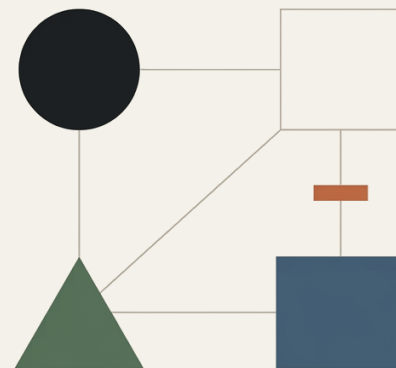
Agent 准入与协同治理基座

跑分测的是模型，不是你的活。

初赛 · 方案 PPT

MIT 开源 | 团队：【请填写】

AgentCorp · Agent 准入与协同治理基座



一页纸速览

为谁做、解决什么、怎么解、凭什么可信。

项目名称

AgentCorp：Agent 准入与协同治理基座

核心解决方案

四个分工明确的 Agent 组成一支「准入评审小组」：拆解任务、出题实测、多维评分、拍板放行。高风险动作必须人工点头才执行。

开放 / 复用价值

MIT 开源。评估层、Skill 内核、可观测投影三者可独立取用；已接入 MCP，可挂接任意标准工具服务。

问题与场景

现有跑分测的是模型的通用能力，不是「它能不能替我做这份工作」；而不同来源的 Agent 之间也没有统一的比较标准。

创新点与差异化

① 同题同标尺的工种实测：所有候选做同一套题，题面内置反注水探针，空话会自己暴露，不看 star 数 ② 审批门真实拦截：高风险动作在人点头前执行不了，不是一句提示语

当前进展

四个 Agent 与五个 Skill 已实现，闭环可离线跑通并导出执行证据；前端 651 项、服务端 282 项自动化测试全绿。

目录

按官方八章组织；右侧百分比为该章对应评分维度的权重。

1	场景与价值	25%	5	工程落地与安全可审计	20%
2	方案总览		6	开放 / 开源计划	5%
3	多 Agent 协同设计	25%	7	落地计划与进展	
4	Skill 工程体系	25%	8	团队介绍 · Demo 视频	

01

场景与价值

对应评分维度 场景价值与行业可复制性

25%

占总评分权重

跑分测的是模型，不是你的活

现有榜单在标准化题库上比通用能力；而真实工作是杂的、有约束的、要交付的。

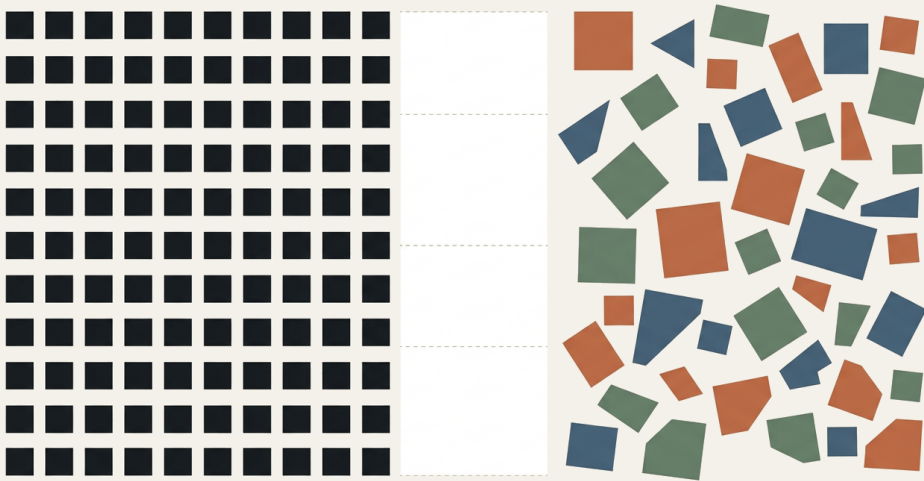
榜单世界

- 统一题库、标准答案、单轮问答
- 衡量的是模型本身的通用能力
- 换个真实任务，结论不一定还成立

真实工作

- 需求模糊、约束具体、要能交付
- 同一个人的活，别人未必这么干
- Agent 是模型+提示+工具+流程的组合

两个世界之间没有桥：跑分高，不代表能替我做这份工作。



更麻烦的是，Agent 之间没有统一的比较标准

不同来源的 Agent 各说各话，选型只能靠试用直觉、star 数或一句「挺好用」。

企业侧 主场景

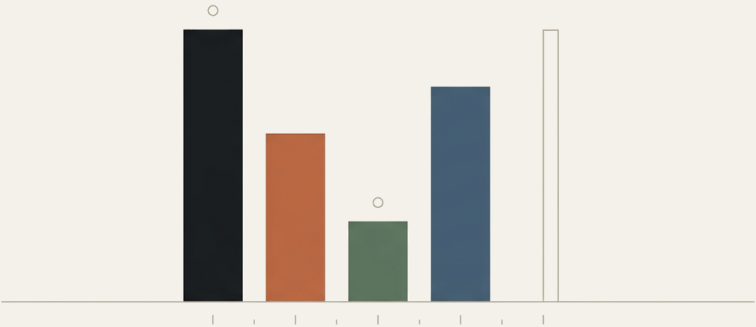
- 要引入外部或自研 Agent 时无从判断
- 答不出：能不能上线、出事谁负责
- 上线之后是否退化，也没人盯

个人侧 延伸场景

- 职场人想找能替自己干活的 Agent
- 只能看 star 数，新作者天然吃亏

两者是同一道题：缺少一把公共的尺子。

AgentCorp 要做的就是这把尺子，以及围绕它的一套准入流程。



把「选型」变成一次有证据的准入评审

像招聘一样：出同一套题、按同一份标准打分、留下可复核的记录、由人拍板。



可量化的能力指标

下列均为可由公开命令复现的系统能力指标，不含未经真实用户验证的业务收益推算。

12 × 3

固定题面 × 工种

k 次全达标

才判定为通过

100%

高风险动作留痕（无绕过路径）

933

项自动化断言

这套流程为什么能跨行业复用

题库可换，流程不变 · 现覆盖写代码 / 写文案 / 做图 · 新增工种只需补题面与评分要点	关卡是共同的 · 任何行业引入 Agent 都要过准入 · 都要回答能不能上线、谁负责	一个引擎，两种用法 · 把「谁在用」作为参数传入评估 · 企业看管理者视角，个人看本人视角
---	--	--

整体架构：谁来判断、用什么能力、连哪些工具、留什么证据

四层各司其职；换掉任何一层的实现，其余三层不受影响。



03

多 Agent 协同设计

对应评分维度 多 Agent 协同与自主闭环能力

25%

占总评分权重

四个分工明确的 Agent，各自的权限写死在代码里

谁能做什么、不能做什么、哪些动作必须人点头——都是程序读取的字段，不是文档描述。

Boss 决策

· 提出需求、设定风险偏好

· 对放行与回滚拍板

· 把结论沉淀成经验

唯一持放行权 · 需人工确认

Dispatcher 编排

· 把任务拆成子任务

· 在成员之间传递上下文

· 追踪状态与轨迹

不可绕过 Boss

Recruiter 出题

· 把岗位要求拆成考察维度

· 组织多轮提问与追问

· 产出面试记录

禁止打分与放行

Evaluator 评分

· 六个维度+工种专项评分

· 重复测量与稳定性检查

· 产出终评报告

禁止放行

交接环节	传递什么	为什么这样设计
Boss → Recruiter	岗位要求与风险偏好	出题必须对准真实需求
Recruiter → Evaluator	面试记录与考察维度	评分只依据实际作答，不看来源
Evaluator → Boss	分数、稳定性、证据	决策者拿到的是证据不是结论

以 AgentTeams 为协同设计基点，且这份映射可以被核对

四张角色卡能一键导出为 AgentTeams（开源实现 HiClaw）的声明式配置，逐项对照迁移成本。

AgentTeams / HiClaw 概念	AgentCorp 对应物	迁移成本
Team 团队	四张角色卡组成的评审小组	协议适配
TeamAdmin 平台管控	Boss（唯一持放行与回滚权）	协议适配
TeamLeader 协作调度	Dispatcher（拆解、传递、监控）	协议适配
Worker 执行体	Recruiter / Evaluator	协议适配
Worker 的技能集	Skill 注册表（含权限校验）	协议适配
Matrix 通信底座	网关消息通道 + 执行轨迹落盘	配置替换
Higress 凭证托管	密钥只存在主进程，前端拿不到	配置替换
控制面 reconcile	当前为进程内编排	需新增工程

这张表不是手写的：由一条命令从角色卡自动导出，产物是可被解析的标准配置文件。

边界：形态已对齐、成本可量化，但尚未在真实控制面上部署——这句话写死在产物首部。

一次完整的准入，会留下八步可复核的记录

下表是真实运行输出，不是示意。每一步都标明了「谁做的」和「用了哪个能力」。

阶段	执行者	调用的能力	输出
接收任务	Boss	—	收到准入申请
任务拆解	Dispatcher	—	拆成 3 步，识别岗位类型为「写代码」
上下文传递	Recruiter	结构化面试	完成实测，交接给评分方
工具调用	Evaluator	多维能力评估	3 次评分全部成功，来源标记为真实裁判
结果验证	Evaluator	可靠性审计	重复测量全部达标，评分未见异常波动
审批	Boss	评审与放行	结论「录用」，但已被审批门拦下，动作未执行
证据沉淀	Dispatcher	—	六步执行轨迹已落盘，可回放
经验沉淀	Boss	评审与放行	最弱项＝产出质量，下一轮定向追问

经验会被复用：同一候选第二次评审时，上一轮的薄弱项会自动进入提问上下文——这一点在实跑输出中可见。

「需要人工确认」不该只是一句提示语

在我们这里，高风险动作在人点头之前，真的执行不了。

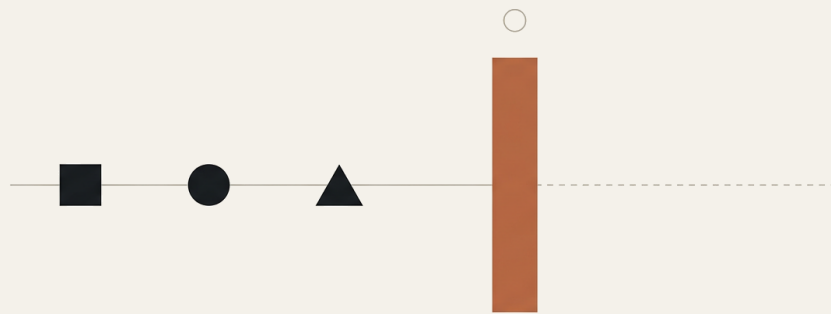
拦下来

- 判定为高风险 → 动作不执行
- 整个流程挂起，等待人工决定
- 人点「放行」之后，动作才发生

撤得回

- 已执行的动作按相反顺序逐条撤销
- 某一条撤不掉，会明确报出来
- 不假装「已完全回滚」

实践证据：判定「录用」之后，录用台账里是 0 条记录——它没敢自己录用。



04

Skill 工程体系

对应评分维度 Skill 工程体系与生态复用

25%

占总评分权重

把反复要做的事，沉淀成能被反复调用的能力

Skill 不是某个 Agent 的一次性行为，而是独立注册、可被任何成员调用、可整体替换的能力单元。

能力单元	谁在用	做什么	失败时怎么办
结构化面试	Recruiter	按考察维度多轮提问并追问	无有效作答就如实交白卷，不替它编
多维能力评估	Evaluator	六个维度+工种专项打分	裁判不可用就标记「未评」，绝不造分
可靠性审计	Evaluator	重复测量，检查结论稳不稳	样本不足则降级，不下硬结论
评审与放行	Boss	综合证据给出处置建议	输入不完整就强制转人工
编排调度	Dispatcher	拆任务、聚合上下文、记轨迹	子任务超时就回写失败，不静默

共同的铁律：任何一个能力单元出问题，都不会让整条流程崩掉，而是带着「我失败了、为什么」继续往下走。

每个能力单元都声明九项契约：用途、输入、输出、触发条件、依赖、失败处理、安全边界、复用价值、协同关系。

单个能力单元长什么样：以「多维能力评估」为例

每个能力单元都按同一份契约声明九项内容，注册时按同一 schema 校验。

名称	多维能力评估（capability_assessment）
用途	对候选 Agent 的实测作答按六个通用维度 + 工种专项维度打分
输入	实测作答记录、使用者画像、面试报告、重复评测次数 k
输出	每次评分结果、均值、综合判定、置信度，以及分数来源标注
触发条件	收到出题方的面试报告后自动触发
依赖	裁判后端、多次评分聚合器、维度注册表
失败处理	裁判超时或报错即标记为「未评」并注明来源，绝不生成推测分数
安全边界	只读作答内容，不修改候选；置信度不足强制标注；降级结果不进经验库
复用价值	维度体系与抗偏差机制可独立用于任何「模型当裁判」的场景

能装上，也能干净地卸下来

能力单元支持注册、覆盖、注销全过程可逆——这是它能被别的项目安全复用的前提。

生命周期

- 注册时返回一个「卸载凭证」
- 卸载即自动清理副作用
- 可临时覆盖，撤销后自动还原

装卸可逆，不留残留

复用方式

- 同类能力可替换实现
- 新增角色无需改动既有代码
- 评估能力可被其他项目单独取用

能力抽象，非行为脚本

接入外部生态

- 已接入 MCP 工具协议客户端
- 支持三种主流连接方式
- 可直接挂接官方与第三方工具

接入是配置，不是改造

工具接入方向	现状	说明
调用外部工具服务	已实现	支持标准 MCP 服务，启动时自动发现可用工具并注入
把自身能力发布为工具服务	后续工程项	接口契约已定义完整，属协议适配而非重新设计

05

工程落地与安全可审计

对应评分维度 工程落地与安全可审计

20%

占总评分权重

让模型当裁判，就得先解决「裁判自己靠不靠谱」

同一份答案让模型连打三次，分数常常不一样。所以我们不采信单次打分。

重复测量，取严不取宽

- 同一份答案独立评多次
- 每一次都达标才算真达标
- 业界称为 $\text{pass}@k$ ：k 次全过才算过，只要有一次掉下来，就不算可靠

打分打飘了要能发现

- 检查多次评分之间的离散程度
- 波动过大就下调置信度并转人工
- 一致性低 = 结论不稳，不是分数低
- 同时检查评分的一致性水平

抵抗常见偏见

- 每轮打乱维度顺序，避免顺序影响
- 明确要求「不因回答长就给高分」
- 评分锚点固定，减少主观漂移

分数来源必须标清楚

- 真实裁判 / 部分降级 / 完全降级 三态
- 降级产生的结论不进入经验库
- 宁可标「未评」，也不填一个数

我们只主张「测得稳」，不主张「测得准」——后者需要长期数据验证，见第七章。

所有数字都能用一条命令复现

验证报告由脚本生成，不是手写；每次改动都会重新跑一遍。

651

前端自动化测试通过

282

服务端测试通过

66

多 Agent 协同专项断言

0

类型与规范检查错误

四条命令跑完全流程

- > pnpm install
- > pnpm web → 打开演示页
- > pnpm verify → 三道门禁 + 出报告
- > pnpm agentteams:export → 导出协同配置

门禁	结果
类型检查（双配置）	通过 · 0 错误
多 Agent 协同专项（9 个文件）	通过 · 66 项断言
隐私扫描	通过 · 零命中
自动生成验证报告	已产出，随代码提交

每一步都留痕，且用的是行业通用字段

执行轨迹按 OpenTelemetry 的生成式 AI 语义规范投影，可接入任意兼容的观测后端。

我们记录的内容	对应的标准字段
执行者名称	gen_ai.agent.name
执行者角色	gen_ai.agent.role
一次运行的关联 id	gen_ai.conversation.id
调用了哪个能力单元	gen_ai.operation.name = invoke_skill
token 用量与成本	gen_ai.usage.input_tokens / output_tokens / cost

检索链路：评估证据只取自本次运行的实测轨迹，不做跨候选检索兜底——避免用别人的表现给当前候选打分。

两条轨迹，同一口径

- 演示闭环：每次运行落一份可回放记录
- 主进程：Agent 之间的委派链另存一份
- 两者可互相投影，成本归因口径一致

边界

- 标准字段已对齐并有测试覆盖
- 接入观测后端为后续工程项

系统出错时的反应，比它顺利时的表现更值得看

每一种失败都有明确的降级方式，且都会如实标注，不会静默吞掉。

出了什么问题	系统怎么做	对结论的影响
裁判模型不可用	标注「未评」，不生成任何分数	结论标记为不可用
多次评分结论不一致	取多数裁决，并按离散程度下调置信度	离散过大则转人工复核
某个能力单元报错	返回失败原因，流程继续	该步标记为告警
越权调用他人能力	直接拒绝并说明边界	阻断，不执行
决策输入不完整	强制转人工，不自动决策	不产出自动结论
回滚时有一条撤不掉	如实报出「只回滚了一部分」	明确提示需人工介入

安全边界

- 密钥只存在于主进程，前端拿不到
- 本地接口逐请求鉴权，未授权直接拒绝
- 题库接口不返回参考答案，防止刷题

云服务选型：必要性与边界

- 裁判走通用协议，可直接指向云端模型服务
- 不绑定单一模型，是为了避免「自己给自己打高分」
- 换服务只改配置，评分逻辑不动

开源的不是一个演示，是三个能被单独取用的部件

任何做 Agent 评估的团队都可以只拿走其中一个，而不必接受整套方案。

评估方法层

· 重复测量与稳定性检查

· 评分偏差与一致性审计

· 抗偏见的评分提示词

适用于任何模型当裁判的场景

能力注册内核

· 可注册、可覆盖、可注销

· 同类能力可替换实现

· 与具体框架解耦

可移植到其他 Agent 框架

可观测投影

· 自有轨迹 → 行业标准字段

· 纯函数、零依赖

· 可接任意兼容的观测后端

不锁定观测供应商

开放项	内容
开源协议	MIT
接口契约	工具调用契约：参数结构、鉴权、错误约定、审计要求、迁移步骤
可核对产物	协同配置清单可一键导出；执行轨迹可回放
第三方依赖	前端 React 生态、桌面端 Electron、服务端 FastAPI；均为各自开源协议，无私有依赖
后续计划	把自身能力发布为标准工具服务；公开评估有效性的验证结果

当前进展：能跑、能验、能复现

初赛不强制提交可运行代码，我们仍然选择用可复现的运行证据说话。

已完成

- 四个 Agent、五个能力单元
- 八步闭环可离线跑通
- 审批门与回滚可用
- 协同配置一键导出
- 外部工具协议已接入
- 651 + 282 项测试全绿

进行中

- 治理层与执行层端到端打通
- 接入标准观测后端
- 扩充工种题库
- 注水对照集纳入持续集成

下一阶段核心目标

- 验证准入评分能否预测真实表现
- 能预测 → 固化维度权重
- 不能预测 → 公开结果并重做
- 把能力发布为标准工具服务

我们承诺公开负面结果：如果某个评分维度对真实交付没有预测力，会明说并调整。

我们没做什么

一个说不清自己边界的评估系统，本身就不值得信任。

边界	实际情况
重复测量的含义	目前测的是「裁判稳不稳」，还不是「Agent 干活稳不稳」
稳 ≠ 准	稳定不等于正确；我们目前只主张前者
协同框架集成	配置形态已对齐，尚未在真实控制面部署
可观测	标准字段已对齐，尚未接入观测后端
适用范围	只用一次、只有一个候选时，这套流程是额外开销
风险控制：高风险动作需人确认；降级结论不入库；自动决策都能回滚并留痕。	



团队介绍

请按官方要求填写：成员背景、分工、过往成果。

【成员一 · 角色】

· 学校 / 公司：【填写】

· 岗位 / 专业：【填写】

· 核心技能：【技术 / 专业领域】

· 本项目分工：【填写】

【成员二 · 角色】

· 学校 / 公司：【填写】

· 岗位 / 专业：【填写】

· 核心技能：【技术 / 专业领域】

· 本项目分工：【填写】

【成员三 · 角色】

· 学校 / 公司：【填写】

· 岗位 / 专业：【填写】

· 核心技能：【技术 / 专业领域】

· 本项目分工：【填写】

项目	内容
团队成果	【过往项目经历 / 获奖经历】
作品集	【作品集或个人主页链接】
代码仓库	【仓库地址】（MIT 开源）

跑分测的是模型，
我们想测的是，它能不能替你做这份工作。

AgentCorp · MIT 开源 | 评估方法、能力内核、观测投影均可单独取用